Random Forest Ensemble Learning for Multiclass Network Intrusion Detection

L. Moodley
Department of Mathematical Sciences
Stellenbosch University
Stellenbosch, South Africa
26186152@sun.ac.za

https://github.com/Leesan0205/MLL4.git

***Abstract*—Network intrusion classification is complicated due to class imbalance, redundant descriptive features and overlapping traffic signatures. An empirical investigation evaluated random forest ensemble learning on the course-specific UNSW-NB15 network traffic dataset. The objectives were to determine whether aggregation improved performance over an individual classification tree and to interpret the effects of ensemble size, feature randomness, tree complexity and class weighting. A stratified development sample supported three-fold parameter selection and controlled experiments. Selected classifiers were subsequently trained on $206,138$ development observations and evaluated on $51,535$ held-out observations. Random forest achieved a test macro-averaged F1 score of 0.6106, compared with 0.5712 for the classification tree and 0.5001 for nearest neighbours. An ensemble of 100 trees was selected as a compromise between validation performance and computational cost. Class weighting raised development macro-F1 and, in a controlled per-class analysis, improved recall on every minority attack category at the cost of precision rather than improving all categories uniformly. A duplicate-aware re-partition that prevented identical feature vectors from spanning the development and test sets did not reduce measured performance, and the ensemble retained its advantage over the single tree, although the extensive feature-vector duplication still limits claims about independent generalisation.**

I. Introduction

Network intrusion detection requires classification of network traffic into normal activity and attack categories. The UNSW-NB15 dataset contains contemporary normal traffic and synthetic attack activity. The version supplied retained ten target classes and substantial class imbalance. The smallest attack category, Worms, comprised 174 of 257,673 observations in the earlier dataset analysis. Such imbalance raises a practical concern: high aggregate accuracy does not guarantee successful detection of infrequent attacks.

An earlier study compared k-nearest neighbours with individual classification trees using the same course dataset. The earlier held-out macro-averaged F1 scores were 0.481 and 0.518. The previous study therefore favoured the classification tree, while also identifying weaknesses for rare attack categories. Ensemble learning builds on this because several perturbed classification trees can be aggregated rather than relying on a single fitted tree. Breiman established that random forests introduce randomness through bootstrap sampling and random predictor selection [4].

Random forest was chosen in preference to a boosting or stacking ensemble so that the comparison isolated the effect of aggregation. The forest retains the classification tree of the earlier study as its base learner and uses the identical encoded feature representation, so a difference in held-out performance is attributable to bootstrap resampling and random predictor selection rather than to a change of model family. This is precisely what the first research question asks. A boosting ensemble would have altered the base-learner fitting procedure in addition to the aggregation rule, and a stacked ensemble would have introduced a second-level learner requiring its own tuning budget; either choice would have confounded that comparison. The design therefore contrasts a similar ensemble against the earlier classification tree, while retaining nearest neighbours as a deliberately dissimilar secondary benchmark.

The present investigation addressed four research questions. The first examined the difference between random forest and a single classification tree. The second investigated changes in predictive performance as the number of trees increased. The third assessed random feature selection during tree construction. The fourth examined whether tree depth, terminal-leaf size and class weighting affected classification of minority attack categories. The principal response variable was macro-averaged F1 because equal class contributions were desirable under the observed class imbalance.

The study used a common train-test partition, model-specific preprocessing and controlled parameter sweeps. The analysis separated development cross-validation from held-out evaluation, with particular attention to the interpretive limitations introduced by duplicate traffic records.

II. Background

A. Classification Trees and Nearest Neighbours

A classification tree partitions a feature space through successive decision rules. A candidate split is selected according to a reduction in node impurity. In the present implementation, the Gini impurity for a node containing C classes was

$$G = 1 - \sum_{c=1}^{C} p_c^2, \tag{1}$$

where p_c denotes the proportion of observations that belong to class c at that node. Recursive splitting creates a model that captures nonlinear relationships without requiring numerical feature standardisation. A small terminal-leaf size allows highly specific rules but increases sensitivity to individual observations. A maximum-depth restriction or larger terminal-leaf size constrains model complexity.

The KNN classifier assigns a class using the labels of nearby training observations. A distance-based classifier is affected by numerical scale, feature redundancy and encoding choices. The earlier study consequently removed redundant or uninformative variables and applied a quantile transformation to numerical features. The nearest-neighbour model serves as a secondary benchmark rather than as an explanation of ensemble gains.

B. Bootstrap Aggregation and Random Forests

Bootstrap aggregation, or bagging, creates multiple training samples by drawing observations with replacement from the available training data. Breiman demonstrated that aggregating unstable classifiers such as decision trees reduced prediction variability in several settings. A random forest extends bagging by restricting the predictors eligible at each split, introducing diversity even when strong predictors occur repeatedly [4].

Let $\hat{p}_b(c \mid \mathbf{x})$ denote the estimated probability of class c from tree b for descriptive-feature vector $\mathbf{x}$, and let B be the number of trees. The implemented classifier averaged class probabilities across the trees,

$$\hat{y}(\mathbf{x}) = \arg \max_{c \in \{1,\ldots,C\}} \frac{1}{B} \sum_{b=1}^{B} \hat{p}_b(c \mid \mathbf{x}), \tag{2}$$

where $\hat{y}$ denotes the predicted class. The variance-reduction benefit depends on both the strength of individual trees and the correlation among their predictions [4]. Increasing B therefore does not guarantee proportional gains. Increasing the number of eligible features can strengthen individual splits while reducing diversity among trees.

C. Imbalance, Regularisation and Expected Effects

Class imbalance changes the effective cost of split decisions. Under unweighted training, improvements for frequent classes can dominate impurity reduction. The `balanced_subsample` option instead computes inverse-frequency class weights separately for each bootstrap sample. He and Garcia reviewed the consequences of imbalanced training distributions and associated evaluation difficulties [5]. Weighted splitting was expected to improve detection of selected rare attacks, although a recall improvement could accompany reduced precision.

Increasing tree depth expands the available decision rules. Unrestricted depth can capture minority patterns but also reproduce noisy labels. Raising the minimum number of observations in a leaf reduces sensitivity to small training groups. The experiments therefore examined whether mild regularisation offered better macro-F1 than fully unconstrained trees. Numerical rescaling was not applied to the tree-based methods because threshold comparisons, unlike nearest-neighbour distances, do not depend on feature magnitudes.

D. Evaluation Measures

The study used precision, recall and the F1 score for each attack class. The class-specific F1 score was

$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c}, \qquad F_{1,\text{macro}} = \frac{1}{C} \sum_{c=1}^{C} F_{1,c}, \tag{3}$$

where P_c and R_c denote precision and recall for class c. Undefined class-specific scores were assigned zero by the evaluation code. Macro-averaging gives every attack category equal influence in the final score. Sokolova and Lapalme analysed how the choice of classification measure affects the interpretation of multiclass results [7]. Accuracy and balanced accuracy were recorded as complementary measures, rather than replacements for macro-F1.

III. Implementation

A. Dataset Preparation

The experiment reused the altered `networkTraffic.csv` supplied for the earlier course assignment rather than substituting an online copy of UNSW-NB15. The file contained 257,673 labelled observations. The original unique identifier was removed. Values greater than one in `is_ftp_login` and `ct_ftp_cmd` were clipped to one, reproducing the documented binary interpretation used in the earlier implementation. Missing `service` values were encoded as an explicit category. Remaining numerical missing values were handled through median imputation fitted within each training fold.

The categorical predictors `proto`, `state` and `service` were one-hot encoded for both the classification tree and random forest. Categories appearing in fewer than 0.1% of training observations were grouped by the encoder. The shared feature representation avoided an artificial ordering among nominal values and supported a more direct comparison between individual trees and their ensemble. Numerical variables were not scaled for either tree-based classifier. The nearest-neighbour pipeline retained the ten exclusions specified in the earlier implementation (`sloss`, `dloss`, `spkts`, `ct_srv_dst`, `ct_src_dport_ltm`, `dwin`, `ct_ftp_cmd`, `stcpb`, `dtcpb` and `response_body_len`) and

TABLE I: Selected model configurations

Model	Final parameters
Random forest	100 trees; half the encoded features per split; unrestricted depth; minimum leaf size 3; bootstrap-specific balanced class weights.
Decision tree	Gini impurity; maximum depth 20; minimum leaf size 1; balanced class weights.
Nearest neighbours	Three neighbours; uniform voting; Euclidean distance.

used a quantile transformation for numerical predictors. Every data-dependent preprocessing operation was included in a fitted pipeline.

B. Classifier Configurations

Python implementations used pandas for data manipulation and scikit-learn for preprocessing, model construction and stratified validation. The random forest used bootstrap sampling and a fixed random seed of 42. Candidate forest configurations differed in the number of eligible features, maximum tree depth, minimum leaf size and class weighting. The earlier classification tree and nearest-neighbour baselines were re-tuned under the revised experiment rather than treated as numerically identical to their earlier configurations.

The final configurations are summarised in Table I. The tree and nearest-neighbour selections differ from Assignment 2 because preprocessing, tuning folds and search procedures were revised. Neither comparison is presented as a controlled reproduction of the earlier numerical results.

IV. Empirical Procedure

A. Data Partitioning and Parameter Search

A stratified 80:20 partition produced 206,138 development observations and 51,535 held-out test observations. A stratified sample of 20,000 development observations supported computationally feasible parameter tuning. Initial random forest tuning evaluated 20 sampled configurations using three-fold stratified cross-validation with 150 trees held fixed. Candidate values were drawn from three feature-selection settings, three depth settings, three leaf-size settings and two class-weighting settings. The principal optimisation measure was mean macro-F1 across validation folds.

After initial tuning, five controlled sweeps each varied a single parameter while holding the others at the best configuration from the initial search. The number of trees was fixed at 150 for every sweep except the ensemble-size sweep itself, matching the initial search rather than the finally selected 100-tree ensemble, because the tree count was chosen only after these sweeps. The number of trees took values 1, 5, 25, 50, 100, 200 and 400. Feature selection compared square-root selection, half of the encoded predictors and all predictors at a split. Depth values were 15, 30 and unrestricted. Leaf sizes were one, three and ten observations. Class weighting was either absent or computed separately for bootstrap samples. The sweep outcomes were interpreted descriptively; fold standard deviations were not treated as formal tests between parameter settings.

A predeclared practical rule selected the smallest ensemble containing at least 25 trees whose mean validation macro-F1 fell within one standard error of the best evaluated ensemble size. The single-tree and five-tree settings were retained as diagnostic ablations. The best initial forest configuration and the size-selection rule determined the final forest parameters in Table I.

TABLE II: Held-out performance of the selected classifiers

Classifier	Macro-F1	Accuracy	Bal. acc.	Fit (s)	Pred. (s)
Random forest	**0.6106**	**0.7976**	**0.6504**	61.7	0.4
Decision tree	0.5712	0.7536	0.6314	2.5	<0.1
Nearest neighbours	0.5001	0.7848	0.4904	1.0	5.5

B. Comparison and Uncertainty

A five-fold stratified development comparison recorded macro-F1 means and sample standard deviations on a 20,000-row sample. However, the sample comprised the same 20,000 stratified development observations used for parameter selection, drawn with the identical random seed, so the diagnostic was not independent of the selection procedure. The resulting cross-validation scores were therefore diagnostic rather than unbiased post-selection estimates. Arlot and Celisse described the need to distinguish cross-validation for model selection from cross-validation used for risk estimation [1].

Each selected model was subsequently fitted using the full development partition and evaluated on the same held-out test partition. Test performance included macro-F1, accuracy, balanced accuracy, class-level precision and recall, and fitting time. A paired stratified bootstrap resampled test observations within their true classes on 250 repetitions. The reported percentile intervals describe uncertainty conditional on the fitted models and the current test partition. They do not quantify variability under a new network environment or a new training sample.

V. Results and Discussion

A. Tuning Outcome and Aggregate Performance

The best sampled forest configuration combined half-feature selection, unrestricted depth, a three-observation minimum leaf size and bootstrap-specific balanced weighting. The initial three-fold search reported validation macro-F1 of 0.5376 and mean training macro-F1 of 0.8149. The large train–validation gap indicated pronounced sensitivity to the training observations. The selected regularisation and weighting settings were therefore interpreted as a balance between fitting rare-class patterns and retaining validation performance.

The random forest achieved the highest held-out macro-F1, accuracy and balanced accuracy, as shown in Table II.

TABLE III: Development cross-validation diagnostics

Classifier	Mean macro-F1	Fold SD
Random forest	0.5284	0.0183
Decision tree	0.4972	0.0198
Nearest neighbours	0.4755	0.0336

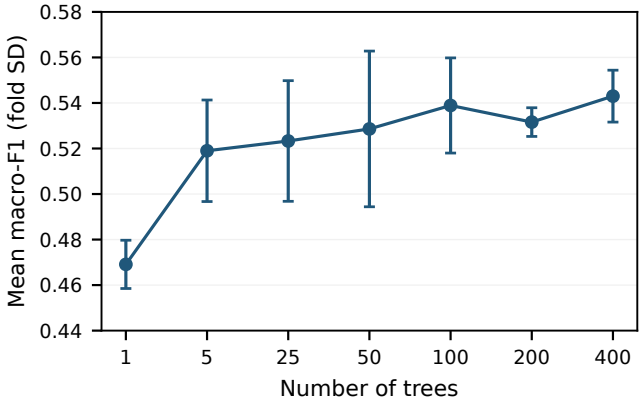


Fig. 1: Effect of ensemble size on development macro-F1. Error bars show fold standard deviations.

Relative to the single tree, the forest increased macro-F1 by 0.0394. The paired bootstrap interval for the difference was [0.0301, 0.0489]. Relative to nearest neighbours, the difference was 0.1105 with an interval of [0.0947, 0.1260]. Neither interval included zero. The evidence consequently favoured the forest for the specific held-out partition, subject to the duplicate-data limitation discussed below.

The diagnostic development comparison in Table III showed the same ranking as the held-out evaluation. The substantially higher final test score for the forest cannot be attributed to ensembling alone. Development cross-validation fitted models on smaller training subsets, whereas the final evaluation used the full set of 206,138 development observations. The observed discrepancy therefore also reflects differences in training-set size and the data-partition characteristics.

B. Ensemble Size

Figure 1 reports the ensemble-size sweep. Mean macro-F1 increased from 0.4691 for a single bootstrap-trained tree to 0.5190 for five trees. The additional gain from five to 100 trees was 0.0199. At 400 trees, macro-F1 reached 0.5430, compared with 0.5389 at 100 trees. Performance was not strictly monotonic: the 200-tree model scored 0.5316. The non-monotonic response is consistent with finite validation samples and changes in prediction votes near decision boundaries, rather than proof that additional trees are systematically harmful.

Mean fitting time rose from 3.26 seconds at 100 trees to 12.31 seconds at 400 trees, an increase of approximately 3.8-fold for a macro-F1 gain of 0.0041. The three-fold standard

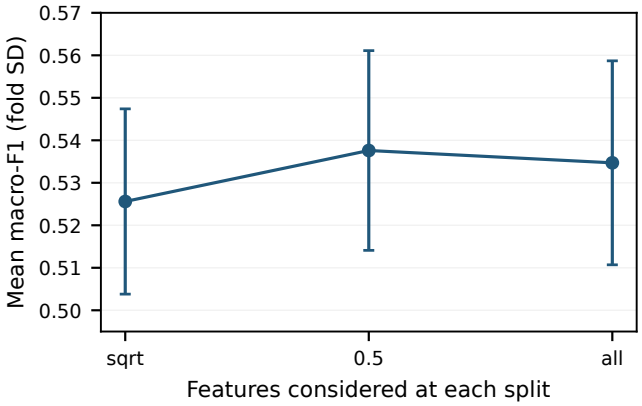


Fig. 2: Effect of candidate feature count at each tree split. Error bars show fold standard deviations.

deviation for the 400-tree setting was 0.0114, giving a standard error of approximately 0.0066. The one-standard-error threshold was therefore approximately 0.5364. The 100-tree result exceeded the threshold, whereas the 50-tree result did not. The selected 100-tree ensemble provided the intended compromise between validation performance and computation.

C. Random Feature Selection

Figure 2 compares the feature-subset settings. Considering half of the encoded predictors at each split produced mean macro-F1 of 0.5376. Square-root selection produced 0.5256, while using all predictors produced 0.5347. The half-feature option therefore exceeded the all-feature configuration by 0.0029 and the square-root configuration by 0.0120.

The result suggests that severe feature restriction weakened individual split decisions for the present feature representation. Using all features also increased the similarity among trees. Half-feature selection appeared to balance these effects. However, standard deviations overlapped substantially and the half-feature advantage over all-feature selection was small. The experiment supports a preference within the tested settings rather than a general claim of statistical superiority. Mean fitting time increased from 1.48 seconds under square-root selection to 4.80 seconds under half-feature selection and 9.31 seconds when every feature was eligible. Half-feature selection consequently also offered a computational compromise.

D. Tree Complexity and Class Weighting

The depth experiment, shown in Figure 3, produced macro-F1 of 0.5447 at a maximum depth of 15. Both depth 30 and unrestricted depth yielded approximately 0.5376. Shallower trees therefore provided a slightly higher mean in the controlled sweep. The improvement was modest relative to the observed fold variability. The result is compatible with restricting overfitting, although the sweep alone did

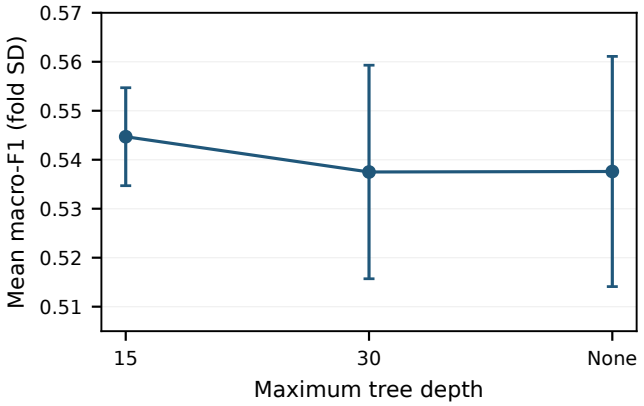


Fig. 3: Effect of maximum tree depth on development macro-F1. Error bars show fold standard deviations.

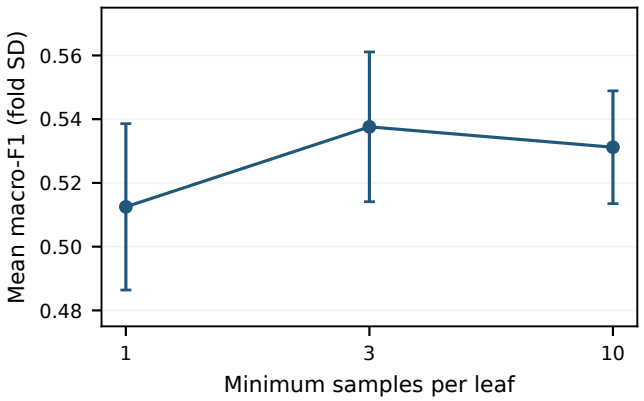


Fig. 4: Effect of minimum leaf size on development macro-F1. Error bars show fold standard deviations.

not establish the cause of the difference. Unrestricted depth was nonetheless retained in the final configuration. The sweeps were one-factor-at-a-time diagnostics around the initial-search optimum, and only the ensemble size was reopened for selection, by a rule stated in advance. The depth-15 advantage of 0.0071 is smaller than the fold standard deviations of the settings compared, so adopting it would amount to selecting on validation noise.

The leaf-size experiment in Figure 4 produced macro-F1 of 0.5125, 0.5376 and 0.5312 for minimum leaf sizes of one, three and ten, respectively. A minimum size of three improved performance over completely unconstrained leaves by 0.0251. Increasing the minimum to ten reduced performance by 0.0064 relative to a leaf size of three. Mild leaf regularisation therefore offered the most favourable balance in the tested settings. The decline at a leaf size of ten is consistent with minority-class decision regions being suppressed by overly broad leaves.

Class weighting produced the largest change among the three regularisation-related sweeps. Figure 5 shows an

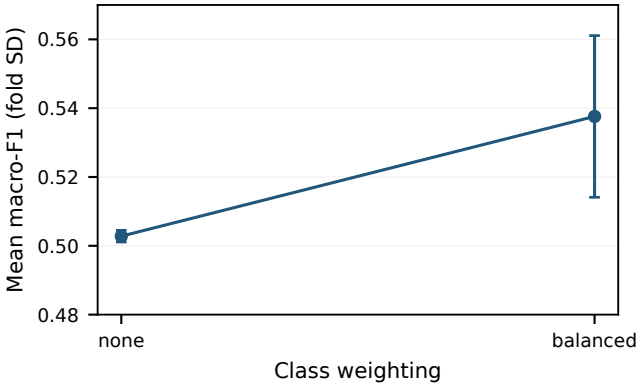


Fig. 5: Effect of bootstrap-specific class weighting on development macro-F1. Error bars show fold standard deviations.

increase from 0.5028 without weighting to 0.5376 with bootstrap-specific balanced weighting, a gain of 0.0348. The interpretation is consistent with higher weighted importance being assigned to minority observations during split selection. The weighted setting also exhibited greater variability across the three folds, with a fold standard deviation of 0.0235 compared with 0.0017 without weighting. The results demonstrate a beneficial effect on the chosen macro-F1 criterion, but not necessarily on every attack class.

Macro-F1 alone does not reveal how weighting redistributes errors among classes. Table IV reports development out-of-fold recall and precision for each class under the two weighting settings, with every other parameter held at the selected configuration. Weighting raised recall for every minority category, most strikingly Worms, from 0.000 to 0.143, together with Backdoor from 0.011 to 0.171, Analysis from 0.024 to 0.101, DoS from 0.327 to 0.459 and Shellcode from 0.615 to 0.701. These gains were offset by reduced recall for the two largest categories, Exploits, from 0.785 to 0.622, and Normal, from 0.932 to 0.887, which is the expected consequence of rebalancing the effective class costs during split selection. Precision moved in the opposite direction for several of the boosted classes: Analysis precision fell from 0.200 to 0.056 and Shellcode from 0.673 to 0.503, indicating that the additional minority detections introduced many false positives, whereas Exploits precision rose from 0.642 to 0.724 as its recall declined. Bootstrap-specific weighting therefore improved the macro-average by trading precision for recall on rare attacks rather than by improving every category uniformly, consistent with the recall–precision trade-off anticipated in the Background.

E. Minority-Class Outcomes

Table V reports held-out recall and F1 for every class under the two tree-based approaches. The table is essential because the macro-average obscures important class-specific trade-offs. Random forest increased Worms recall

TABLE IV: Controlled per-class effect of bootstrap-specific class weighting (development out-of-fold; all other parameters fixed at the selected configuration)

	Recall		Precision	
Class	None	Weighted	None	Weighted
Analysis	0.024	0.101	0.200	0.056
Backdoor	0.011	0.171	0.083	0.082
DoS	0.327	0.459	0.355	0.347
Exploits	0.785	0.622	0.642	0.724
Fuzzers	0.547	0.654	0.630	0.593
Generic	0.974	0.974	0.997	0.999
Normal	0.932	0.887	0.900	0.932
Reconnaissance	0.729	0.771	0.897	0.809
Shellcode	0.615	0.701	0.673	0.503
Worms	0.000	0.143	0.000	0.500

from 0.771 to 0.943 and Worms F1 from 0.614 to 0.767. The corresponding Shellcode recall increased from 0.758 to 0.848, with F1 increasing from 0.583 to 0.690. The forest also increased recall for Normal traffic from 0.802 to 0.916.

The improvement was not universal. Backdoor recall decreased from 0.300 to 0.176, while Backdoor F1 declined from 0.072 to 0.062. Analysis recall decreased from 0.217 to 0.153 and Analysis F1 decreased from 0.104 to 0.081. These values provide evidence that broad gains in macro-F1 did not resolve the most difficult class boundaries. The poor precision for Backdoor and Analysis under the forest, approximately 0.038 and 0.055 respectively, also indicates substantial false-positive assignment to these classes.

Worms performance requires particular caution. The held-out test contained only 35 Worms observations. A recall of 0.943 corresponded to approximately 33 correctly identified observations. A change of a small number of predictions would consequently change the reported recall considerably. Strong performance on such a small class is encouraging but insufficient evidence of stable detection in a different traffic environment. Table IV isolates the effect of weighting at fixed values of every other parameter, whereas Table V describes the final weighted forest against the single tree; the two must not be conflated, because the latter comparison also changes the aggregation rule.

F. Computational Trade-offs and Validity

The forest required 61.7 seconds to fit on the full development dataset, compared with 2.5 seconds for the classification tree. The approximately 25-fold fitting-time increase is the computational cost associated with the observed predictive improvement. The nearest-neighbour method trained in 1.0 second but incurred its cost at prediction, as Table II shows: labelling the 51,535 held-out observations required 5.5 seconds, against 0.4 seconds for the forest and under 0.1 seconds for the single tree. Nearest-neighbour classification defers all computation to inference, so despite the fastest training it was by an order of magnitude the slowest to predict. Training time alone therefore misstated deployment cost. A complete recommendation would additionally require memory-footprint comparisons, because the nearest-neighbour model must retain the full development set at inference time, whereas the two tree-based models store only fitted decision rules.

The data audit identified 23,401 held-out observations with feature vectors identical to at least one development observation, representing approximately 45.4% of the test partition. Feature-identical observations can lead to optimistic estimates when their labels and traffic contexts are closely related. The current split was stratified by attack class, not grouped by repeated traffic signature or network session. The paired bootstrap also resampled the existing test partition and could not correct the underlying dependence. The numerical intervals must consequently be interpreted as conditional evidence about the selected test data rather than independent deployment-performance guarantees.

A second limitation arose from the small tuning subsample. Twenty thousand observations provided computational efficiency but represented the rarest attack categories with few examples. Three-fold tuning consequently produced noisy class-level estimates, particularly for Worms. Furthermore, feature selection, parameter tuning and the diagnostic development comparison reused overlapping observations. The development cross-validation means in Table III were not independent post-selection measurements. A group-aware evaluation that keeps identical feature vectors within a single partition is reported in Section V-G; repeated training seeds and minority-class confidence intervals remain useful further checks.

G. Duplicate-Aware Robustness

The 45.4% feature-vector overlap motivated a direct test of whether it inflated the held-out comparison. Each observation was assigned to a group defined by a hash of its descriptive feature vector, and an 80:20 partition was drawn with grouping enforced, so that rows sharing a feature vector were always placed in the same partition. The resulting split contained no feature vector common to both sides. The 257,673 observations formed 153,684 distinct feature vectors, and the partition produced 206,355 development and 51,318 test observations. The three selected configurations from Table I were refitted and evaluated without any re-tuning, so that any change was attributable to the partitioning rule rather than to a different model.

Table VI shows that removing cross-partition duplicates did not reduce measured performance. Macro-F1 was stable for nearest neighbours and slightly higher for both tree-based models, so the overlap had not inflated the stratified estimates in the direction of concern. More importantly for the first research question, the forest retained its advantage over the single tree: the margin was 0.0424 under the group-aware partition against 0.0394 under the stratified partition, and the model ranking was unchanged. The

TABLE V: Held-out attack-category recall and F1 for the two tree-based classifiers

Class	Test support	Forest recall	Forest F1	Tree recall	Tree F1
Normal	18,600	0.916	0.929	0.802	0.875
Reconnaissance	2,798	0.784	0.840	0.788	0.819
Backdoor	466	0.176	0.062	0.300	0.072
DoS	3,271	0.444	0.374	0.430	0.388
Exploits	8,905	0.571	0.684	0.518	0.646
Analysis	535	0.153	0.081	0.217	0.104
Fuzzers	4,849	0.689	0.689	0.748	0.625
Worms	35	0.943	0.767	0.771	0.614
Shellcode	302	0.848	0.690	0.758	0.583
Generic	11,774	0.979	0.989	0.981	0.986

TABLE VI: Macro-F1 under stratified and duplicate-aware (group-aware) partitioning for the identical selected configurations

Classifier	Stratified	Group-aware	Change
Random forest	0.6106	0.6347	+0.0242
Decision tree	0.5712	0.5923	+0.0211
Nearest neighbours	0.5001	0.4999	−0.0002

principal conclusion is therefore robust to the duplicate-traffic concern.

The small increases should not be read as a de-duplication benefit. The group-aware partition is not stratified, so the class proportions and the test set itself differ from the stratified split, and the two macro-F1 values are not computed on the same observations. The shift in balanced accuracy for the single tree, from 0.6314 to 0.6700, confirms that the test composition changed. The experiment supports the stability of the ranking and of the ensemble advantage rather than a precise numerical equivalence between partitions, and absolute scores should still be interpreted as conditional on the evaluation data.

VI. Conclusions

The investigation evaluated random forest ensemble learning for ten-class network intrusion classification. The forest outperformed the re-tuned individual classification tree on the held-out macro-F1 criterion by 0.0394. The performance gain was accompanied by stronger detection of Worms and Shellcode, but the Backdoor and Analysis categories remained poorly distinguished. Ensemble learning therefore improved aggregate class-balanced performance without uniformly improving each attack category.

Increasing the number of trees produced large initial gains and diminishing subsequent returns. A 100-tree ensemble was selected by the stated one-standard-error rule. Half-feature selection provided the highest mean validation performance among the investigated candidate-feature settings, although its advantage over using all predictors was small. Mild leaf regularisation improved the development score. Bootstrap-specific balanced class weighting increased mean macro-F1 by 0.0348 compared with no weighting in the controlled sweep.

The random forest was the preferred model for the evaluated split when macro-F1 was prioritised over training and prediction cost. A duplicate-aware re-partition that prevented identical feature vectors from spanning the development and test sets left the model ranking unchanged and preserved the forest's advantage over the single tree, which strengthens the comparison against the concern of duplicated traffic. A controlled per-class analysis further showed that bootstrap-specific weighting improved minority-class recall while reducing precision for the same categories. The principal remaining limitations are the small number of rare-attack observations and the reliance on a single training seed and test partition; repeated seeds and minority-class confidence intervals would further test performance in independent network traffic.

Acronyms and Symbols

Acronyms, in alphabetical order: DoS—denial of service; F1—harmonic mean of precision and recall; kNN—k-nearest neighbours; UNSW-NB15—name of the University of New South Wales network traffic dataset.
Symbols, in alphabetical order: B—number of trees; C—number of classes; $F_{1,c}$—F1 score of class c; G—Gini impurity; p_c—node-level class proportion; P_c—precision of class c; R_c—recall of class c; $\hat{p}_b$—estimated class probability from tree b; $\mathbf{x}$—feature vector; $\hat{y}$—aggregated predicted class.

References

[1] S. Arlot and A. Celisse, "A survey of cross-validation procedures for model selection," *Statistics Surveys*, vol. 4, pp. 40–79, 2010, doi: 10.1214/09-SS054.

[2] G. Biau and E. Scornet, "A random forest guided tour," *TEST*, vol. 25, no. 2, pp. 197–227, 2016, doi: 10.1007/s11749-016-0481-7.

[3] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, pp. 123–140, 1996, doi: 10.1007/BF00058655.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[5] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009, doi: 10.1109/TKDE.2008.239.

[6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proceedings of the 2015 Military Communications and Information Systems Conference*, pp. 1–6, 2015, doi: 10.1109/MILCIS.2015.7348942.

[7] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009, doi: 10.1016/j.ipm.2009.03.002.